

Desenvolvimento de Site para Escala

<https://gemini.google.com/app/5809c9c1d044ba8d>

User prompt: Gemini preciso desenvolver um site para fazer as escalas do grupo de acolitos da minha paróquia. Esse site servira para demonstrar as escalas das missas tanto as missas semanais quanto as missas de fim de semana. A ideia principal é que o site permita o cadastro dos acolitos e de forma automática e randomica ele crie as escalas para as celebrações com algumas regras de negocio: O site deve permitir tres classificações de acolitos, "maiores", "menores" e responsaveis. Cada celebração deve ter um responsavel e uma mescla de maiores e menores. Também gostaria que o site me permitisse colocar dois acolitos para servirem sempre juntos (pois temos alguns que são irmãos e assim fica mais facil) além disso queria algo que pudesse ver o endereço do acolito e o endereço da igreja para ver qual fica mais viavel para ele servir. Me ajude a pensar como esse site deve ser construido, qual linguagem, e oque mais pode ter passado despercebido por mim ou voce não tenha entendido

Response: Que excelente iniciativa! Automatizar as escalas de coroinhas e acólitos resolve uma dor gigantesca em quase toda paróquia, pois fazer isso na mão sempre gera confusão, esquecimentos e dá muito trabalho.

A sua lógica base está muito bem pensada. As regras de negócio (níveis de experiência, líderes, irmãos servindo juntos e geolocalização) formam um ótimo escopo inicial.

Aqui estão os pontos que você deve considerar, desde o que pode ter passado despercebido até as tecnologias ideais para tirar isso do papel.

1. O que pode ter passado despercebido (Regras e Cenários)

Quando criamos sistemas que lidam com pessoas, precisamos prever as exceções. Considere adicionar estas funções ao seu planejamento:

- **Gestão de Indisponibilidade (Bloqueio de Agenda):** Os acólitos ficam doentes, viajam ou têm provas. O sistema precisa permitir que o acólito (ou os pais) marquem dias específicos em que eles *não podem* servir, para que o sorteio automático os ignore nessas datas.
- **Balanceamento Justo:** O algoritmo randômico puro pode acabar sorteando o "João" três vezes no mês e o "Pedro" nenhuma. O sistema precisa registrar quantas vezes cada um serviu no mês para priorizar quem serviu menos.
- **Aprovação e Edição Manual:** O computador vai montar a escala, mas você (ou o coordenador) precisa poder olhar, editar e fazer trocas manuais antes de publicar. Às vezes, o sistema acerta a regra, mas você sabe que dois acólitos específicos não se dão bem ou que alguém pediu uma troca de última hora.
- **Quantidades por Missa:** O sistema precisa saber exatamente quantas vagas existem por celebração. Por exemplo: Missa de Domingo às 8h precisa de 1 Responsável, 2 Maiores e 3 Menores.
- **Múltiplas Comunidades (Igrejas):** Como você mencionou a distância até a igreja, imagino que a

paróquia tenha uma Igreja Matriz e capelas. É preciso cadastrar cada igreja e suas respectivas missas.

2. Como construir: Tecnologias e Linguagens

Existem dois caminhos principais para criar esse site. Você pode programar do zero (código tradicional) ou usar plataformas visuais que criam sistemas sem precisar escrever código (No-Code).

Caminho A: Programação Tradicional

Se o objetivo for programar o site do zero, esta é uma combinação excelente e muito usada no mercado:

Parte do Site	Tecnologia Recomendada	Por que escolher?
Backend (O Cérebro)	Python (com framework Django)	Python é fantástico para cálculos matemáticos e regras complexas. A lógica de cruzar distâncias, checar irmãos e sortear pessoas fica muito mais fácil de escrever nele.
Frontend (O Visual)	React.js ou Vue.js	Criam interfaces modernas, rápidas e que funcionam muito bem tanto no computador quanto no navegador do celular.
Banco de Dados	PostgreSQL	Muito robusto, gratuito e perfeito para conectar os dados (Acólito -> Missa -> Escala).
Mapas e Distância	Google Maps API	O Google fornece uma ferramenta (Distance Matrix API) que calcula automaticamente a distância real entre o endereço A e o B.

Caminho B: Plataformas No-Code (Recomendado para agilidade)

Se você quer focar em resolver o problema rápido sem precisar se aprofundar meses em linguagens de programação complexas, as plataformas **No-Code / Low-Code** são o caminho ideal.

Você constrói o site arrastando blocos e configurando as regras visualmente.

- **Glide:** Excelente para criar aplicativos (que funcionam como sites no celular) a partir de planilhas. É muito rápido para sistemas de gestão interna e escalas.
- **Bubble:** É a ferramenta visual mais poderosa do mercado hoje. Permite criar todo o sistema de login, as regras complexas de sorteio e o banco de dados em um só lugar.

3. A Lógica de Sorteio (O coração do sistema)

Quando for criar a automação, ela não pode ser um sorteio 100% cego. O sistema deve seguir uma **ordem de prioridade** ao montar cada missa:

1. **Filtro de Disponibilidade:** Exclui quem marcou que não pode ir.
 2. **Filtro de Localização:** Seleciona os acólitos que moram num raio de "X" km daquela igreja específica.
 3. **Seleção do Responsável:** Sorteia 1 pessoa com a tag "Responsável" (priorizando quem serviu menos no mês).
 4. **Encaixe de Irmãos:** Se um dos irmãos for sorteado para a vaga de "Maior" ou "Menor", o sistema automaticamente puxa o outro para preencher a vaga seguinte.
 5. **Preenchimento:** Sorteia o restante dos "Maiores" e "Menores" necessários para fechar as vagas daquela missa.
-

User prompt: Quero um plano de desenvolvimento com uma ideia mais tecnica. Vamos escrever os requisitos e daí partimos para um plano documental. Não será necessário o sistema de gestão de indisponibilidades, o balanceamento justo é completamente valido, o modo de validação manual por parte do coordenador também é necessario, o sistema deveria permitir o cadastro de igrejas e celebrações para essas igrejas pois sim possui a matriz e suas capelas. Requisitos: Forma de "login" para o coordenador fazer separações manuais. Cadastro de acolitos com as tags "menor", "maior" e "responsavel". Sorteio justo, todos os acolitos devem servir mais ou menos a mesma quantidade de vezes durante o mês. Geolocalização, comparando o endereço do acolito com as igrejas proximas para escalar para uma missa proxima de suas residencia (o sistema deve permitir desativar essa opção). Revise a ideia e me ajude a pensar se falta mais alguma coisa.

Response: Excelente decisão. Fazer um "corte" de funcionalidades (como retirar o sistema de indisponibilidade) para focar no que é essencial é uma das melhores práticas no desenvolvimento de software. Isso cria o que chamamos de MVP (Produto Mínimo Viável) — entregamos valor rápido e melhoramos depois.

Como você pediu uma visão mais técnica, estruturei a sua ideia no formato de um Documento de Requisitos padrão. Isso serve como um "mapa de construção", organizando exatamente o que o sistema fará.

Abaixo, dividi a sua ideia em Requisitos Funcionais (o que o sistema faz) e Regras de Negócio (como o sistema toma decisões).

1. Documento de Requisitos (Versão Inicial)

Requisitos Funcionais (RF) - As funcionalidades do sistema

ID	Funcionalidade	Descrição Técnica
RF01	Autenticação	Tela de login exclusiva para o Coordenador (Administrador) acessar o painel de controle.
RF02	Gestão de Locais	Cadastro de Igrejas (Matriz e Capelas), registrando Nome e Endereço Físico completo.
RF03	Gestão de Missas	Cadastro das Celebrações vinculadas a uma Igreja, definindo dia da semana, horário e a quantidade exata de vagas para cada tag (ex: 1 Responsável, 2 Maiores, 2 Menores).
RF04	Gestão de Acólitos	Cadastro dos servidores contendo: Nome, Endereço e as Tags obrigatórias (Maior, Menor, Responsável).
RF05	Motor de Sorteio	Algoritmo que gera a escala de um período (ex: um mês) aplicando as Regras de Negócio.
RF06	Edição Manual	Interface onde o coordenador visualiza a escala gerada pelo sistema e pode remover/adicionar acólitos livremente antes de salvar a versão final.
RF07	Toggle de Geolocalização	Um "interruptor" (botão de ligar/desligar) no painel do coordenador que define se o Motor de Sorteio deve usar a distância como critério.

Regras de Negócio (RN) - As leis que o sistema deve obedecer

- **RN01 - Balanceamento Justo:** O sistema deve armazenar um histórico (ou contador) de quantas vezes cada acólito serviu no mês atual. O algoritmo de sorteio deve priorizar ordenar a fila de escolha pelos acólitos com o menor número no contador.
- **RN02 - Composição Obrigatória:** Nenhuma missa pode ser gerada sem preencher a vaga do "Responsável".
- **RN03 - Cálculo de Distância (Geolocalização):** Se o *Toggle* (RF07) estiver ativo, o sistema deve converter os endereços em coordenadas (Latitude/Longitude) e priorizar a alocação do acólito na Igreja mais próxima de sua residência.

2. O que pode estar faltando (Revisão Técnica)

Analisando a estrutura acima, trago alguns pontos técnicos para refinarmos juntos:

1. **A Regra dos Irmãos:** No seu planejamento anterior, você mencionou colocar irmãos para servirem juntos. Essa regra não apareceu na sua última lista. Nós vamos mantê-la? (T tecnicamente, isso exige criar um vínculo entre dois cadastros no banco de dados).
2. **Como a escala chega até os acólitos?**
 - Como não haverá login para os acólitos, o sistema precisa gerar uma página pública (um link como escala.com/matriz/agosto) que você possa copiar e mandar no grupo do WhatsApp para que eles apenas visualizem.
3. **A "Mágica" da Geolocalização:**
 - Para o sistema comparar distâncias, ele não consegue ler "Rua X". Ele precisa transformar o endereço digitado no cadastro em coordenadas de mapa (Latitude e Longitude) usando algo chamado *Geocoding* (como a API do Google Maps). Isso é super viável, mas exige que, no momento do cadastro do acólito, o endereço seja digitado corretamente (com CEP e número).
4. **Zerar o Contador:**
 - Para o balanceamento justo funcionar, o sistema precisa saber quando "virar o mês". O sorteio justo avalia o histórico total ou a contagem zera a cada 30 dias?

O próximo passo técnico seria desenhar a estrutura do Banco de Dados (como as tabelas se conectam). Como você prefere lidar com a regra dos irmãos e com a forma de os acólitos visualizarem a escala finalizada?

User prompt: 1. Gostaria de implementar sim a regra dos irmãos. 2. A ideia seria hospedar o site em alguma ferramenta como o vercel e que a pagina inicial do site seja a escala ja visivel para que ao acessar o link do vercel o acolito veja a escala. 3. Podemos deixar os campos ja separados e obrigatorios na hora do cadastro como tipo da via (rua, avenida, estrada), logradouro, numero e cep. 4. Gostaria que contador zerace ao virar o mês. Agora queria que o sistema analise o calendario de festas e solenidades da igreja catolica, nem que eu tenha que fazer na mao as datas e as comemorações. Mas a ideia central é que o sistema quando for criar a escala do mês ele analise essa tabela do banco e caso no dia específico tenha uma festa ou solenidade ou ate mesmo que o registro no banco diga que a missa sera em um horario diferente e tera uma quantidade diferente de acolitos, o sistema siga oque esta la. analise e veja se é viavel

Response: Essa sua nova ideia sobre o calendário litúrgico é fantástica. Na verdade, ela transforma o seu sistema de um simples "sorteador" para uma ferramenta de gestão paroquial real. Festas, Solenidades e Tríduos (como a Semana Santa) sempre quebram a rotina normal, e prever isso no código é uma sacada excelente.

Respondendo diretamente aos seus 4 pontos e à nova funcionalidade: é **100% viável**.

Aqui está como essas novas regras se encaixam no nosso plano técnico e como elas vão funcionar na prática:

1. Resolução dos 4 Pontos

- **A Regra dos Irmãos (Vínculo):** No banco de dados, criaremos uma conexão entre os perfis. Quando o motor de sorteio for rodar, ele não vai sortear o "João" e depois ver se o "Pedro" cabe. Ele vai sortear o "Bloco João + Pedro". Se a missa tiver vagas para os perfis dos dois, ambos entram. Se não tiver, o motor pula os dois e procura outra missa onde ambos caibam.
- **Vercel e Página Inicial:** Escolha perfeita. O Vercel é gratuito para projetos assim e extremamente rápido. O site não terá uma "tela de boas-vindas" inútil. Ao acessar `escala-paroquia.vercel.app`, a tela principal já será a tabela com a escala do mês vigente. Apenas se você acessar `/admin` é que o sistema pedirá senha.
- **Endereço Estruturado:** Separar em (Tipo da via, Logradouro, Número, CEP, Bairro) é a melhor prática de banco de dados. Isso garante que a API de geolocalização não se perca com abreviações ou erros de digitação.
- **Contador Mensal:** Muito simples de implementar. Toda vez que você clicar em "Gerar Escala de [Mês/Ano]", o sistema vai ignorar o histórico de meses anteriores e contar apenas as missões dentro daquele mês específico para fazer o balanceamento.

2. A Nova Funcionalidade: Calendário Litúrgico e Exceções

A forma mais inteligente de programar isso é usar o conceito de **Padrões vs. Exceções**.

Como vai funcionar no Banco de Dados: Você terá duas tabelas para gerenciar as missas:

1. **Missas Fixas (A Rotina):** Aqui você cadastra o padrão. *Ex: Todo Domingo, 08h e 19h, na Matriz. Vagas: 1 Resp, 2 Maiores, 2 Menores.*
2. **Calendário de Solenidades (As Exceções):** Aqui você cadastra datas específicas. *Ex: 15 de Agosto (Assunção de Nossa Senhora), 19h30, na Matriz. Vagas Especiais: 2 Resp, 4 Maiores, 4 Menores.*

Como o Motor de Sorteio vai agir: Quando você mandar o sistema gerar a escala de Agosto, ele seguirá 3 passos cronológicos antes de sortear os acólitos:

1. **Mapeamento:** Ele preenche todos os domingos e dias de semana normais usando as regras das *Missas Fixas*.
2. **Sobreposição (Override):** Ele olha a tabela de *Solenidades*. Se no dia 15 de Agosto já houvesse uma missa fixa cadastrada, o sistema "apaga" a regra fixa daquele dia e **substitui** pela regra da Solenidade (horário diferente, mais acólitos).
3. **Sorteio:** Com os "moldes" dos dias perfeitamente desenhados, ele começa a puxar os acólitos usando as regras de irmãos, proximidade (se ativa) e balanceamento.

3. Atualização dos Requisitos

Para o documento técnico, adicionamos:

- **RF08 - Visão Pública:** A rota raiz do site (`/`) exibirá a escala do mês gerada, otimizada para leitura em telas de celular, sem necessidade de login.

- **RF09 - Calendário de Exceções:** Painel para o administrador cadastrar datas específicas (Solenidades/Eventos), definindo horário e quantidade de vagas personalizadas, que terão prioridade sobre as missas fixas.
- **RN04 - Vinculação de Irmãos:** O algoritmo deve agrupar acólitos vinculados e tentar alocá-los obrigatoriamente na mesma celebração, desde que haja vagas compatíveis para os cargos de ambos.

Com a arquitetura no Vercel, a melhor linguagem para construir tudo isso de forma unificada (Frontend e Backend juntos) é o **Next.js (React) usando TypeScript**.

User prompt: Como ficaria a estrutura das tabelas do banco de dados para suportar todas essas regras? Pode me mostrar as conexões entre Acólitos, Missas e as Escalas?

Response: Com essa estrutura bem definida, o banco de dados pode ser desenhado usando o modelo Relacional (SQL), como o **PostgreSQL**. A lógica para fazer tudo funcionar de forma eficiente é separar o banco em três blocos: **Cadastros, Regras (Os Moldes) e Ação (A Escala Real)**.

Aqui está o desenho de como as tabelas (e suas conexões) vão ficar:

Bloco 1: Os Cadastros (Entidades Físicas)

1. Tabela: igrejas Guarda os dados da matriz e das capelas.

- **id** (Chave Primária)
- **nome_igreja** (Ex: Matriz São José, Capela Santa Rita)
- *tipo_via, logradouro, numero, cep, bairro* (Para o Google Maps ler)
- **lat / lng** (Coordenadas de latitude e longitude salvas automaticamente)

2. Tabela: acolitos O cadastro dos servidores.

- **id** (Chave Primária)
- **nome_completo**
- **classificacao** (Opções: Responsável, Maior ou Menor)
- **grupo_familiar_id** (Número inteiro. Se o João e o Pedro são irmãos, ambos recebem o mesmo ID aqui. Se for um acólito único, fica vazio. É assim que o sistema amarra eles no sorteio).
- *tipo_via, logradouro, numero, cep, bairro*
- **lat / lng** (Coordenadas da casa dele para comparar com as da igreja)

Bloco 2: As Regras (Os "Moldes" das Missas)

A chave do sucesso do seu sistema está aqui. Em vez de criar missas uma a uma na mão, você cria "moldes".

3. Tabela: missas_padrao (A Rotina)

- **id** (Chave Primária)
- **igreja_id** (Chave Estrangeira -> Conecta com a Tabela 1)

- **dia_semana** (Ex: 0 para Domingo, 6 para Sábado)
- **horario** (Ex: 08:00)
- **vagas_responsavel** (Ex: 1)
- **vagas_maior** (Ex: 2)
- **vagas_menor** (Ex: 3)

4. Tabela: **calendario_excecoes** (As Festas e Solenidades)

- **id** (Chave Primária)
- **igreja_id** (Chave Estrangeira -> Conecta com a Tabela 1)
- **data_exata** (Ex: 15/08/2026 - Assunção de Nossa Senhora)
- **horario** (Ex: 19:30)
- **titulo_evento** (Ex: "Missa Solene da Padroeira")
- **vagas_responsavel** (Ex: 2)
- **vagas_maior** (Ex: 4)
- **vagas_menor** (Ex: 4)
- **substitui_rotina?** (Sim/Não. Se 'Sim', apaga a missa normal do dia. Se 'Não', cria uma missa extra naquele dia).

Bloco 3: A Ação (A Escala Gerada)

Quando você clica em "Gerar Escala de Agosto", o sistema lê o **Bloco 2**, e "carimba" essas informações no **Bloco 3**. É o Bloco 3 que os acólitos vão visualizar no Vercel.

5. Tabela: escalas_geradas (A Celebração Específica) O sistema cria uma linha para cada missa real que vai acontecer no mês.

- **id** (Chave Primária)
- **igreja_id** (Conecta com a Tabela 1)
- **data_hora** (A data e hora exata, ex: 02/08/2026 às 08:00)
- **titulo** (Ex: "18º Domingo Comum" ou "Missa da Padroeira")
- **mes_ano_referencia** (Ex: "08-2026" - Fundamental para zerar o contador do mês)

6. Tabela: escala_acolitos (O Cruzamento / Conexão) Esta é uma tabela de relacionamento (Muitos-para-Muitos). É ela que diz *quem* vai estar em *qual* missa.

- **id** (Chave Primária)
- **escala_id** (Conecta com a Missa na Tabela 5)
- **acolito_id** (Conecta com o Acólito na Tabela 2)

Como essa estrutura resolve os seus problemas na prática?

1. **Como o Contador Mensal zera automaticamente?** O sistema não precisa zerar nada ativamente no banco. Quando ele for sortear a próxima missa do mês de Agosto (08-2026), o algoritmo só precisa fazer a seguinte pergunta ao banco: *"Banco, conte quantas vezes o Acólito ID 5 aparece na Tabela 6 (escala_acolitos) onde a missa for do mes_ano_referencia 08-2026."* Se for setembro, o filtro procura por 09-2026. Logo, o João começa Setembro com ZERO missões cadastradas.
2. **Como o Sistema liga os irmãos?** Antes de sortear as pessoas, o algoritmo olha quem falta. Precisa de 2 maiores? Ele pega um acólito aleatório. É o João. O João tem o grupo_familiar_id = 14. O sistema imediatamente procura na tabela de acólitos quem mais tem o ID 14 (o Pedro). Se o Pedro também for "Maior" e houver vaga, o sistema escala os dois juntos numa tacada só.

3. **Como a edição manual do Coordenador funciona?** Se você quiser trocar um acólito pelo painel, tudo o que o site faz é deletar uma linha da **Tabela 6** e criar outra no lugar apontando para o Acólito novo que você escolheu. O resto do banco fica intacto.

Com esse banco desenhado, o "chassi" do seu sistema está pronto.

User prompt: Entendi a estrutura do banco. Como vai funcionar matematicamente o algoritmo para cruzar a localização da casa do acólito com a igreja e depois priorizar quem tem menos missas?

Response: Para que o motor de sorteio funcione perfeitamente, o algoritmo precisa de um sistema de "pontuação" (ou pesos) para tomar decisões sozinho. Ele não fará escolhas aleatórias cegas; ele fará **escolhas aleatórias dentro de regras matemáticas rígidas**.

A matemática por trás disso se divide em duas partes: o cálculo da distância no globo terrestre e a ordenação (ranking) dos acólitos.

Veja como a lógica de código deve ser construída:

1. A Matemática da Distância (Fórmula de Haversine)

Como a Terra é esférica, você não pode simplesmente subtrair uma coordenada da outra. Na programação, usamos uma equação matemática chamada **Fórmula de Haversine**, que calcula a menor distância entre dois pontos (Latitude e Longitude) considerando a curvatura do planeta.

A boa notícia: você não precisa escrever essa equação complexa do zero. Se você usar o banco de dados **PostgreSQL**, ele possui uma extensão chamada **PostGIS**. Você apenas envia o comando: Calcule a distância entre a Igreja A e o Acólito B. O banco processa a matemática e devolve, em milissegundos, a distância exata em metros.

2. O Algoritmo de Prioridade (Ranking)

Para cada vaga na missa, o sistema cria uma lista de todos os acólitos disponíveis daquela classificação (ex: todos os "Maiores") e dá notas para eles.

A ordenação dessa lista seguirá uma hierarquia estrita:

1. **Critério 1 (O Rei): Menor número de missas no mês.** (Quem tem 0 missas *sempre* ganha de quem tem 1).
2. **Critério 2 (O Desempate Técnico): Menor distância.** (Se o botão de geolocalização estiver ligado).
3. **Critério 3 (A Justiça): Ordem Aleatória.** (Para evitar que o sistema sempre escale os mesmos acólitos em caso de empate nos critérios 1 e 2).

3. O Algoritmo em Ação (Passo a Passo)

Vamos simular que o sistema está gerando a escala para uma missa na Capela Santa Rita, que precisa de **2 Acólitos Menores**. O algoritmo roda o seguinte ciclo:

Passo 1: O Filtro Inicial O sistema pega todos os acólitos marcados como "Menores". (Temos o João, o Pedro, o Lucas e o Tiago).

Passo 2: A Contagem e o Cálculo O sistema olha para o banco de dados do mês vigente e calcula a distância até a Capela:

- **João:** 1 missa servida | 2 km de distância
- **Pedro:** 0 missas servidas | 5 km de distância
- **Lucas:** 0 missas servidas | 3 km de distância
- **Tiago:** 0 missas servidas | 3 km de distância

Passo 3: O Ranking (A Mágica) O sistema ordena a lista aplicando os 3 critérios: 1º lugar: **Lucas** (0 missas, 3km) - *Empatou com o Tiago, mas o critério aleatório o colocou primeiro.* 2º lugar: **Tiago** (0 missas, 3km) 3º lugar: **Pedro** (0 missas, 5km) - *Perdeu para Lucas e Tiago na distância.* 4º lugar: **João** (1 missa, 2km) - *É o mais perto, mas já serviu no mês, então vai pro fim da fila.*

Passo 4: A Regra dos Irmãos (A Exceção de "Fura-Fila") O sistema puxa o 1º da lista: **Lucas**. A primeira vaga está preenchida. Imediatamente, o algoritmo verifica o grupo_familiar_id do Lucas.

- *O Lucas tem irmão?* Sim, o João é irmão dele.
- *O João é Acólito Menor?* Sim.
- *Ainda tem vaga para Menor nessa missa?* Sim, falta 1 vaga.

A Ação do Fura-Fila: O algoritmo ignora temporariamente o ranking, puxa o **João** (mesmo ele estando em último) e preenche a segunda vaga. Assim, os irmãos servem juntos e as duas vagas são preenchidas. O Tiago e o Pedro ficam para a próxima missa.

Por que esse formato é o ideal?

Porque ele é "à prova de falhas". Se a geolocalização for desativada no painel, o sistema simplesmente ignora o Critério 2 e desempata tudo entre o Critério 1 (Contador mensal) e o Critério 3 (Aleatoriedade). Ninguém fica sobrecarregado e as igrejas sempre têm a equipe completa.

User prompt: Quais telas e funções precisam existir no painel administrativo do Coordenador para ele gerenciar isso tudo de forma fácil?

Response: O segredo de um bom painel administrativo é focar na usabilidade. Como é extremamente comum assumir a gestão dessas ferramentas tendo pouquíssimo conhecimento técnico sobre como os algoritmos funcionam por trás das cortinas, a interface precisa ser visual, direta e à prova de erros, sem jargões de programação. O coordenador precisa entrar, clicar em botões claros e ver o resultado.

Para que a gestão seja fluida, o painel do Coordenador deve ser dividido em **5 telas principais**:

1. Visão Geral (Dashboard Inicial)

É a primeira tela que o coordenador vê ao fazer login. Ela serve como um "raio-X" rápido do mês.

- **Alertas:** Avisos como "Faltam 3 acólitos para a missa de domingo às 19h" ou "A escala do próximo mês ainda não foi gerada".
- **Atalhos Rápidos:** Botões grandes para "Gerar Nova Escala" ou "Cadastrar Acólito".

2. Gestão de Acólitos (A Equipe)

Uma tela focada em quem serve, com uma lista simples e um botão de "Adicionar Novo". Ao clicar no acólito, abre-se um formulário com:

- **Dados Pessoais e Endereço:** Campos separados para o motor de geolocalização funcionar perfeitamente.
- **Classificação:** Um menu para escolher entre *Responsável*, *Maior* ou *Menor*.
- **Vínculo Familiar:** Um campo visual de busca. Ex: você digita "Pedro", seleciona o perfil dele e o sistema cria a regra de irmãos automaticamente, sem o coordenador precisar mexer em IDs ou códigos.

3. Locais e Rotinas (A Estrutura)

Onde as regras fixas moram.

- **Aba Igrejas:** Cadastro da Matriz e das Capelas (com seus endereços).
- **Aba Missas Fixas:** Uma tabela semanal. Você escolhe a Igreja, o dia, o horário e define o "molde" de vagas (quantos Maiores, Menores e Responsáveis).

4. Calendário Litúrgico (As Exceções)

Uma tela em formato de calendário (parecida com o Google Agenda).

- Se o coordenador clica no dia 15 de agosto, abre uma janela perguntando: "*Qual a solenidade? Qual o horário? Quantas vagas?*".
- Uma caixa de marcação simples: "*Esta missa substitui a missa normal desse dia?*". Se marcado, o sistema já sabe o que fazer.

5. A Fábrica de Escalas (Onde a mágica acontece)

Esta é a tela mais importante de todo o sistema. Ela é dividida em duas etapas:

- **Etapas 1: A Geração**
 - Um menu simples para escolher o mês e o ano (Ex: Setembro / 2026).
 - Um botão de "Ligar/Desligar Geolocalização".
 - Um botão grande e destacado: "**Gerar Escala**".
- **Etapas 2: A Revisão (Edição Manual)**
 - Após o carregamento, a tela não publica a escala direto. Ela mostra um rascunho visual das semanas.
 - **Troca Fácil:** Se o algoritmo escalou o "João" com o "Pedro" e o coordenador sabe que os dois juntos fazem bagunça, ele clica no nome do João. Um menu suspenso (dropdown) se abre mostrando apenas os acólitos compatíveis (da mesma classificação) que não estão escalados naquele horário, para uma substituição rápida e segura.
 - Botão final: "**Aprovar e Publicar**". Só depois desse clique a escala vai para o link público no Vercel.

Dessa forma, o sistema faz o trabalho pesado de cruzamento de dados, mas o poder de decisão e o refinamento final ficam totalmente nas mãos do coordenador, através de cliques simples.

Quer detalhar como documentar isso tudo de forma técnica para entregar a um programador e iniciar a construção, ou prefere pensar na parte das notificações pelo WhatsApp?

User prompt: creio que inicialmente ja esta bom, como faço para gerar um documento disso com os requisitos e as regras de negocio?

Response: Consolidar tudo em um único documento é o passo definitivo para tirar a ideia do papel. Este formato funciona como uma ponte perfeita: ele traduz a sua ideia de forma lógica para quem vai programar, permitindo que você gerencie o projeto e guie a criação do sistema com total segurança, mesmo sem dominar a parte técnica da escrita de códigos.

Você pode copiar o texto abaixo e colar em um Word ou Google Docs para usar como seu documento oficial de projeto.

Documento de Especificação: Escala de Acólitos

1. Visão Geral do Sistema

Plataforma web dividida em duas áreas: uma interface pública hospedada para visualização rápida das escalas por parte dos acólitos, e um painel administrativo restrito para a coordenação gerenciar pessoas, locais, exceções litúrgicas e acionar o algoritmo de sorteio automatizado.

2. Requisitos Funcionais (O que o sistema faz)

ID	Funcionalidade	Especificação Técnica
RF01	Painel Administrativo	Acesso restrito por login com 5 telas de gestão (Dashboard, Acólitos, Locais, Calendário Litúrgico e Motor de Sorteio).
RF02	Gestão Estrutural	Cadastro de igrejas (matriz/capelas com endereço) e "moldes" de missas fixas definindo a quantidade exata de vagas por cargo.

ID	Funcionalidade	Especificação Técnica
RF03	Perfil de Acólitos	Cadastro contendo endereço completo (para geocodificação), classificação (Maior, Menor, Responsável) e vínculo familiar.
RF04	Motor de Sorteio	Algoritmo acionado manualmente que gera o rascunho da escala mensal cruzando as regras de negócio. Possui botão para ligar/desligar o uso de geolocalização.
RF05	Edição e Publicação	Interface visual para o coordenador revisar a escala, fazer trocas manuais via menu suspenso e publicar a versão final.
RF06	Visão Pública	Rota raiz do site que exhibe a tabela da escala do mês vigente, otimizada para dispositivos móveis, sem necessidade de login.

3. Regras de Negócio (Como o sistema decide)

- **RN01 - Balanceamento Justo:** A ordenação para o sorteio sempre prioriza os acólitos com o menor número de atuações. O contador de missões de cada acólito deve zerar automaticamente na virada do mês de referência.
- **RN02 - Geolocalização:** Quando ativo, o algoritmo utiliza o endereço estruturado para calcular a menor distância real entre a residência e a igreja, usando isso como principal critério de desempate.
- **RN03 - Vínculo de Irmãos:** Ao sortear um acólito, o sistema deve verificar seu ID de grupo familiar e forçar o encaixe do irmão na mesma celebração, desde que existam vagas compatíveis com sua classificação.
- **RN04 - Exceções Litúrgicas:** Eventos e solenidades cadastrados no calendário do sistema têm prioridade absoluta, sobrepondo e substituindo a estrutura de vagas das missas fixas daquele dia específico.

Você quer que eu crie um roteiro de perguntas estratégicas para você usar quando for entrevistar o programador que vai desenvolver isso?

User prompt: Crie um documento word com os requisitos funcionais, as regras de negocio e o modelo de

Response: Vou gerar o documento completo com todas as especificações técnicas, requisitos, regras e o modelo do banco de dados que estruturamos.

Como o formato padrão aqui é texto, estou gerando um arquivo estruturado em Markdown. Você pode facilmente copiar todo o texto dentro do arquivo gerado e colar diretamente em um documento do Microsoft Word ou Google Docs, e a formatação de tabelas e tópicos será mantida.

O arquivo está pronto! Este documento contém toda a "planta baixa" da sua aplicação. Com esse nível de detalhe estruturado em requisitos e tabelas de banco de dados, qualquer programador ou equipe de desenvolvimento entenderá exatamente o escopo do projeto, evitando custos extras e atrasos.

Como sugestão, basta copiar e colar isso em um arquivo Word e salvar em PDF para enviar como proposta ou escopo técnico! Posso ajudar com mais algum detalhe do planejamento?

User prompt: Gemini, vamos fazer assim. Eu vou te passar uma visão geral e, daí, quero que você me diga se está de acordo com esse plano ou se precisa adicionar ou retirar algo e me ajude a pensar também se está tudo certo, se está faltando alguma coisa ou não, porque eu preciso terminar essa documentação para começar esse projeto logo. Então, a ideia do sistema é a seguinte: uma, um site onde vão ser apresentadas as escalas dos acólitos de uma paróquia católica. Ou seja, eu preciso de uma aba principal que, ao abrir o site, que vai ser hospedado na Vercel, vai ser usado o banco de dados PostgreSQL do Supabase e eu pretendo desenvolver ele em Python com o FastAPI. Então, a ideia é: a primeira tela com as escalas já aparecendo de cara, porque a ideia é que o acólito não precisa mexer em nada, ele não tenha menus para mexer, literalmente não tenha nada. E, daí, tem apenas a tela onde vai mostrar e um botão de login. Nesse botão de login, o coordenador entra com o seu login próprio e, ali dentro, ele pode fazer as as gerência, né, das escalas. A ideia é: uma aba com listas, onde eu vou poder cadastrar, vou poder alterar, vou poder fazer um CRUD com os acólitos; uma outra aba que eu possa fazer um CRUDzinho com igrejas, eu possa cadastrar igrejas e, dentro dessas igrejas, eu possa cadastrar missas, ali aquela ideiazinha dos modelos das missas. Eu tenho uma ideia também de uma outra aba com calendário para exceções. Além disso, eu queria agora que eu conseguisse fazer, denominar para o meu sistema que, por exemplo, qual domingo, qual semana do tempo comum eu estou e ele vai seguindo ali sem eu ter que alterar nada. É, que me vem à cabeça é agora, faça uma análise disso que eu falei, veja se o documento tem tudo isso, se tem coisas a mais me mostre, porque eu posso ter esquecido de falar, se tem coisas a menos me fale também e me ajude a pensar se está faltando algo ou se está sobrando algo.

Response: O plano está extremamente sólido e a escolha da arquitetura é excelente. Utilizar o FastAPI para lidar com as regras de negócio complexas, o Supabase gerenciando o PostgreSQL e o frontend servido pelo Vercel forma uma stack moderna, rápida e perfeita para esse tipo de aplicação full-stack.

Fazendo o cruzamento entre o que você acabou de descrever e a documentação que montamos anteriormente, temos o seguinte cenário:

1. O que está perfeitamente alinhado (Sincronizado)

- **Visão Pública (Zero Fricção):** A ideia da Vercel abrir direto na tabela de escalas, sem menus ou distrações para o acólito, está garantida no **RF06**. É a melhor abordagem de usabilidade.
- **Login e CRUDs:** As abas exclusivas para o coordenador gerenciar Acólitos, Igrejas e as Missas (os

moldes fixos) batem perfeitamente com os **RF01, RF02 e RF03**.

- **Calendário de Exceções:** A aba para as solenidades e quebras de rotina também já está desenhada no **RN04**.

2. O que você não mencionou agora (Mas está no documento e não pode ser esquecido)

Ao descrever as telas, você listou os cadastros (CRUDs), mas deixou de fora **a tela principal de ação e as regras invisíveis**:

- **A Tela de Geração de Escala (RF04 e RF05):** Faltou mencionar a tela onde o coordenador efetivamente escolhe o mês, aperta o botão "Gerar Escala", visualiza o rascunho criado pelo sistema, faz os ajustes manuais e clica em "Publicar". Essa é a tela que aciona o FastAPI para fazer os cálculos.
- **As Regras de Sorteio (RN01, RN02 e RN03):** Não se esqueça de que no backend você precisará programar a lógica de balanceamento justo (quem serviu menos), a regra de colocar irmãos juntos (agrupamento familiar) e o cruzamento de geolocalização.

3. A Grande Novidade: O Tempo Litúrgico Automático

Essa é uma adição brilhante que **não estava** no documento original e eleva bastante o nível técnico do projeto.

Fazer o sistema saber se estamos no "3º Domingo do Tempo Comum" ou no "2º Domingo do Advento" de forma automática exige um cuidado especial na programação. O calendário litúrgico católico não é fixo (como o calendário civil); ele é móvel e orbita ao redor da data da Páscoa (que depende dos ciclos lunares) e do Natal (que é fixo).

Como resolver isso no FastAPI: Você não precisará cadastrar isso na mão ano após ano. Como você vai estruturar o backend em Python, você tem dois caminhos elegantes para essa funcionalidade:

1. **Consumir uma API externa:** Existem APIs abertas e gratuitas (como a *calapi.inadiutorium.cz*) que fornecem o calendário litúrgico católico romano completo em formato JSON. O seu backend só precisa consultar a data atual e a API devolve a cor litúrgica, a semana e o tempo.
2. **Biblioteca Python:** Você pode usar bibliotecas específicas de cálculo de datas móveis (algoritmo de *Computus* para a Páscoa) para calcular o ano litúrgico inteiro dinamicamente direto no seu servidor.

O que precisa ser atualizado na Documentação?

Para que você possa começar a codar com a documentação 100% redondinha, precisamos fazer duas pequenas adições ao escopo:

1. **Adicionar Requisitos Não Funcionais (Stack Tecnológica):** Oficializar no documento que o backend será em Python (FastAPI), banco PostgreSQL (Supabase) e hospedagem Frontend na Vercel (provavelmente com React/TypeScript, que casa perfeitamente com esse ecossistema).
 - *Dica de infraestrutura:* Lembre-se de que a Vercel é fantástica para o frontend, mas hospedar APIs em Python nela como *serverless functions* pode ter algumas limitações de tempo de execução. Vale considerar hospedar o FastAPI no Render ou Railway, deixando a Vercel apenas com a interface.
2. **Adicionar o RF07 - Sincronização Litúrgica:** "O sistema deve identificar e exibir automaticamente o tempo litúrgico atual e a respectiva semana (ex: Tempo Comum, Quaresma) na interface de escalas,

baseando-se no calendário litúrgico romano anual."

A base do projeto está completa. O escopo não está "gordo" demais (o que atrasaria o desenvolvimento), nem simples demais (o que não resolveria o problema real da paróquia).

User prompt: Por favor, reescreva a documentação final incluindo as escolhas de tecnologia (FastAPI, Supabase, Vercel), a tela de geração e a nova regra de cálculo do ano litúrgico, para eu fechar esse arquivo.

Response: Documentação de Especificação: Sistema de Escalas Paroquiais

1. Visão Geral e Arquitetura Tecnológica

Plataforma web projetada para automatizar a criação e gestão de escalas de acólitos em uma paróquia (matriz e capelas). O sistema possui uma interface pública de acesso direto para leitura das escalas e um painel administrativo restrito para a coordenação.

- Frontend:** Construído em TypeScript (React/Next.js) e hospedado na **Vercel**. Focado em responsividade e performance.
- Backend:** Construído em Python utilizando o framework **FastAPI**. Responsável pelo processamento do algoritmo de sorteio, cálculo de distâncias matemáticas e lógicas litúrgicas.
- Banco de Dados:** **PostgreSQL** hospedado no **Supabase**, utilizando modelo relacional para garantir a integridade das conexões entre acólitos, igrejas e missas.

2. Requisitos Funcionais (O que o sistema faz)

ID	Funcionalidade	Especificação Técnica
RF01	Visão Pública (Zero Fricção)	A rota raiz do site (/) exibe diretamente a escala do mês vigente. Não possui menus de navegação ou ações para o acólito, apenas a tabela otimizada para leitura em celulares e um botão discreto para login da coordenação.
RF02	Autenticação Administrativa	Tela de login restrita. Ao autenticar, o coordenador acessa o painel de controle (Dashboard) com métricas

ID	Funcionalidade	Especificação Técnica
		rápidas e atalhos de gestão.
RF03	Gestão Estrutural (CRUD)	Módulos independentes para o cadastro e edição de Igrejas (Matriz e Capelas, com endereço físico) e Missas Padrão (definindo dia da semana, horário e os "moldes" exatos de vagas para Maiores, Menores e Responsáveis).
RF04	Gestão de Acólitos (CRUD)	Módulo para cadastrar os servidores contendo: Nome, Endereço completo (para geocodificação), Classificação (Maior, Menor, Responsável) e um campo visual de Vínculo Familiar para conectar irmãos.
RF05	Calendário de Exceções	Interface para cadastrar solenidades, tríduos ou eventos especiais, definindo a data exata, horário e uma estrutura de vagas personalizada que foge da rotina.
RF06	Fábrica de Escalas (Geração e Edição)	Tela onde o coordenador seleciona o Mês/Ano e aciona o motor de sorteio. A tela deve apresentar o <i>rascunho</i> gerado pelo algoritmo, permitir trocas manuais rápidas (via menu suspenso filtrando acólitos compatíveis) e possuir o botão final de "Publicar Escala" .
RF07	Sincronização Litúrgica	O backend (FastAPI) deve identificar dinamicamente a semana e o tempo litúrgico atual (ex: 3º Domingo do Tempo Comum, 1º Domingo

ID	Funcionalidade	Especificação Técnica
		do Advento) utilizando cálculos de datas móveis ou consumo de API litúrgica externa, enviando essa informação para exibição automática no cabeçalho das missas na interface pública.

3. Regras de Negócio (Como o sistema decide)

- **RN01 - Balanceamento Justo Mensal:** O motor de sorteio deve priorizar os acólitos com o menor número de atuações. O contador de missões servidas deve contabilizar apenas os registros do mês vigente (ex: "08-2026"), garantindo que todos iniciem o mês subsequente zerados para o algoritmo.
- **RN02 - Vínculo Familiar (Irmãos):** Ao alocar um acólito que possua um grupo_familiar_id preenchido, o algoritmo suspende momentaneamente o ranking padrão e busca obrigatoriamente alocar os irmãos vinculados na mesma celebração, desde que existam vagas pendentes compatíveis com suas classificações.
- **RN03 - Geolocalização via Haversine (Opcional):** Se ativado pelo coordenador na tela de geração, o algoritmo deve cruzar as coordenadas (Lat/Lng) do acólito com as da Igreja utilizando cálculo matemático (via PostGIS ou lógica no backend) para usá-la como principal critério de desempate técnico na alocação.
- **RN04 - Sobreposição de Exceções (Override):** As datas cadastradas no Calendário de Exceções têm prioridade absoluta. Se houver conflito de datas, o sistema deve anular o "molde" da missa padrão daquele dia e utilizar a estrutura de horários e vagas definida na exceção.

4. Modelo de Banco de Dados (PostgreSQL / Supabase)

Bloco 1: Cadastros Físicos

Tabela: igrejas

- id (PK, UUID)
- nome_igreja (String)
- endereco_completo (String - tipo_via, logradouro, numero, cep, bairro)
- lat_lng (Coordenadas para cálculo de distância)

Tabela: acolitos

- id (PK, UUID)
- nome_completo (String)
- classificacao (Enum: 'Responsavel', 'Maior', 'Menor')
- grupo_familiar_id (Integer - ID compartilhado entre irmãos para agrupamento)
- endereco_completo (String)
- lat_lng (Coordenadas)

Bloco 2: Regras e Moldes

Tabela: missas_padrao (A Rotina)

- id (PK, UUID)
- igreja_id (FK -> igrejas.id)
- dia_semana (Integer - 0 a 6)
- horario (Time)
- vagas_responsavel (Integer)
- vagas_maior (Integer)
- vagas_menor (Integer)

Tabela: calendario_excecoes (Solenidades)

- id (PK, UUID)
- igreja_id (FK -> igrejas.id)
- data_exata (Date)
- horario (Time)
- titulo_evento (String - Ex: "Solenidade de Corpus Christi")
- vagas_responsavel, vagas_maior, vagas_menor (Integers)
- substitui_rotina (Boolean - True para apagar a missa padrão do dia)

Bloco 3: Ação e Publicação

Tabela: escalas_geradas (A Celebração Instanciada)

- id (PK, UUID)
- igreja_id (FK -> igrejas.id)
- data_hora (Timestamp)
- tempo_liturgico (String - Ex: "3º Domingo do Advento" - preenchido automaticamente via FastAPI)
- mes_ano_referencia (String - Ex: "08-2026" - usado para zerar o contador do balanceamento)
- status (Enum: 'Rascunho', 'Publicado')

Tabela: escala_acolitos (O Relacionamento N:N)

- id (PK, UUID)
- escala_id (FK -> escalas_geradas.id)
- acolito_id (FK -> acolitos.id)

User prompt: ótimo vou iniciar o desenvolvimento a partir disso. para um bom fluxo de desenvolvimento devo criar o banco primeiro correto? Segunda coisa, consigo desenvolver tudo no VSCode sem baixar nada a mais?

Response: Sim, o fluxo ideal é começar exatamente pelo banco de dados. Como toda a aplicação gira em torno de cruzar informações (acólitos, missas, regras), construir as tabelas no PostgreSQL via Supabase primeiro cria o "contrato" oficial dos dados. Com a base estruturada, fica muito mais ágil criar os modelos de dados no backend em FastAPI e, posteriormente, desenhar as telas no frontend.

Sobre o VSCode: ele será sua central de comando e você escreverá 100% do código nele, mas o editor sozinho não "roda" os projetos. Para desenvolver e testar essa stack localmente, você precisará baixar e instalar três ferramentas fundamentais no seu computador:

1. **Node.js:** É o motor necessário para rodar o ambiente do frontend (React/Next.js) e compilar o TypeScript.
2. **Python:** O interpretador necessário para rodar o seu backend e levantar o servidor local do FastAPI.
3. **Git:** O sistema de versionamento de código. Ele é obrigatório para conectar o seu código ao GitHub. É através dessa conexão que a Vercel captura seu repositório e cria os pipelines de deploy automático.

Caso você queira isolar os ambientes sem instalar o Node.js e o Python diretamente no sistema operacional do seu computador, você pode optar por criar containers Docker para rodar o backend e o frontend, mapeando as portas diretamente para o VSCode. No entanto, para começar rápido, a instalação tradicional das três ferramentas acima é o caminho mais direto.

User prompt: Pode me passar os comandos de terminal para eu criar as pastas do projeto, inicializar o backend em FastAPI e o frontend em Next.js dentro do VSCode?

Response: Com certeza. Como vamos estruturar um projeto *full-stack*, a melhor prática é criar uma pasta principal (raiz) e, dentro dela, colocar o frontend e o backend separados.

Abra o seu terminal (pode ser o PowerShell) e siga este passo a passo para montar a estrutura base e já abrir tudo no VSCode:

1. Criar a pasta raiz do projeto

Este comando cria a pasta principal e já abre o VSCode nela:

```
mkdir escala-paroquia
cd escala-paroquia
code .
```

2. Inicializar o Backend (FastAPI)

Agora, dentro do terminal integrado do VSCode (Ctrl + ' ou Ctrl + ``), vamos criar a pasta do backend, montar o ambiente virtual (para isolar as bibliotecas) e instalar o FastAPI.

```
# Cria a pasta e entra nela
mkdir backend
cd backend
```

```
# Cria o ambiente virtual
python -m venv venv
```

```
# Ativa o ambiente virtual (se estiver no Windows)
.\venv\Scripts\Activate
```

```
# Instala o FastAPI, o servidor Uvicorn e o cliente do Supabase
pip install fastapi uvicorn supabase python-dotenv

# Cria o arquivo principal do servidor
New-Item main.py

# Volta para a pasta raiz
cd ..
```

Observação: Se o PowerShell der erro de script bloqueado ao ativar o venv, rode Set-ExecutionPolicy Unrestricted -Scope CurrentUser.

Para testar se funcionou: Abra o arquivo backend/main.py no VSCode e cole este código básico:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"mensagem": "Servidor da Paróquia rodando!"}
```

Para rodar, no terminal (dentro da pasta backend com o venv ativado), digite: `uvicorn main:app --reload`. Acesse `http://localhost:8000` no navegador.

3. Inicializar o Frontend (Next.js)

Abra uma **nova aba** no terminal do VSCode (para manter o backend rodando) e certifique-se de que está na pasta raiz `escala-paroquia`. Vamos rodar o instalador do Next.js:

```
npx create-next-app@latest frontend
```

O terminal fará algumas perguntas. Recomendo responder assim para casar com o nosso escopo:

- *Would you like to use TypeScript?* **Yes**
- *Would you like to use ESLint?* **Yes**
- *Would you like to use Tailwind CSS?* **Yes** (Vai ajudar muito no visual rápido)
- *Would you like to use src/ directory?* **Yes**
- *Would you like to use App Router?* **Yes** (Padrão mais novo e moderno)
- *Would you like to customize the default import alias?* **No**

Para testar se funcionou:

```
cd frontend
npm run dev
```

Acesse `http://localhost:3000` no navegador e você verá a tela inicial do Next.js.

Pronto! Seu ambiente está 100% configurado. Você tem o banco desenhado, a documentação pronta e as duas aplicações rodando localmente.

User prompt: tive alguns erros que nao me permitiram rodar a aplicação em python me ajude a resolver

Response: Esses dois erros são clássicos de configuração inicial de ambiente e estão relacionados aos caminhos das pastas e à forma como o VSCode lê o seu projeto.

1. Erro no Terminal: Could not import module "main" (Referente à image_a5136a.png)

O problema aqui é a pasta onde você está executando o comando. Olhando o caminho no seu terminal, você está na raiz do projeto (...\\escala-paroquia>), mas o arquivo `main.py` foi criado dentro da pasta `backend`. O `uvicorn` tentou procurar o arquivo na raiz e não encontrou.

Como resolver: No seu terminal, navegue para a pasta correta antes de iniciar o servidor:

```
cd backend
uvicorn main:app --reload
```

2. Erro no VSCode: Import "fastapi" could not be resolved (Referente à image_a5138f.png)

Esse aviso amarelo do Pylance acontece porque o editor de texto do VSCode está usando o Python global do seu Windows para tentar entender o código, em vez de usar o Python do ambiente virtual (`venv`) onde você realmente instalou o FastAPI.

Como resolver: Você precisa avisar o VSCode qual é o interpretador correto:

1. Com o VSCode aberto, aperte **Ctrl + Shift + P** para abrir a paleta de comandos.
2. Digite **Python: Select Interpreter** (ou *Python: Selecionar Interpretador*) e aperte Enter.
3. Na lista que aparecer, procure e clique na opção que aponta para o seu ambiente virtual. O caminho será algo parecido com `\\.backend\\venv\\Scripts\\python.exe` (geralmente ele ganha uma tag escrita ('venv': venv) ao lado).

Assim que você selecionar o interpretador correto, o VSCode vai reconhecer a biblioteca e a linha amarela de erro desaparecerá em poucos segundos.

User prompt: ótimo tudo funcionando agora. Quais os proximos passos?

Response: O próximo passo natural no desenvolvimento full-stack é estruturar o banco de dados real e conectá-lo ao seu servidor FastAPI. Como toda a lógica e os cruzamentos do sistema dependem das

tabelas, o Supabase é a nossa prioridade agora.

Aqui está o roteiro prático para essa fase:

1. Criar o Projeto no Supabase

1. Acesse o site do Supabase (supabase.com), faça login (pode ser com o GitHub) e crie um **New Project**.
2. Defina um nome (ex: escala-paroquia) e gere uma senha segura para o banco de dados (guarde essa senha).
3. Após o projeto carregar (leva alguns minutos), vá no menu lateral em **Project Settings** (ícone de engrenagem) > **API**.
4. Copie duas informações cruciais: a **Project URL** e a chave **anon / public**.

2. Criar o arquivo de variáveis de ambiente (.env)

Nunca colocamos chaves de banco de dados diretamente no código. Dentro da pasta backend no VSCode, crie um arquivo chamado exatamente .env e cole os dados que você copiou do Supabase:

```
SUPABASE_URL=cole_a_sua_project_url_aqui
SUPABASE_KEY=cole_a_sua_chave_anon_aqui
```

3. Conectar o Python ao Supabase

Vamos atualizar o seu arquivo backend/main.py para que ele leia as chaves secretas do .env e estabeleça a conexão com o banco toda vez que o servidor iniciar. Substitua o código atual por este:

```
import os
from fastapi import FastAPI
from dotenv import load_dotenv
from supabase import create_client, Client

# Carrega as variáveis de ambiente do arquivo .env
load_dotenv()

# Puxa as credenciais
SUPABASE_URL = os.getenv("SUPABASE_URL")
SUPABASE_KEY = os.getenv("SUPABASE_KEY")

# Inicia a conexão com o banco
supabase: Client = create_client(SUPABASE_URL, SUPABASE_KEY)

app = FastAPI()

@app.get("/")
def read_root():
    return {"status": "Servidor rodando", "banco_conectado": supabase is not None}
```

Se o seu terminal ainda estiver rodando o `uvicorn`, ele vai reiniciar sozinho. Ao acessar `localhost:8000`, você deve ver a mensagem confirmando a conexão.

4. Criar as Tabelas no Banco (O Esquema)

No painel do Supabase, você usará a ferramenta **SQL Editor** (ícone de código no menu lateral). É lá que construímos o banco de dados executando comandos SQL para gerar as tabelas de Igrejas, Acólitos e Missas exatamente como desenhamos na documentação.

Quer que eu gere o script SQL completo com todas as tabelas prontas para você apenas copiar, colar e rodar lá no Supabase?

User prompt: são essas as chaves?

Response: Sim, essas são exatamente as chaves!

O Supabase atualizou recentemente o layout das chaves de API, dividindo entre **Publishable key** (que substitui a antiga chave anon) e **Secret keys** (que substitui a antiga chave service_role).

Para o seu projeto no FastAPI, você vai usar:

1. **SUPABASE_URL**: Você encontra logo acima ou na aba de configurações gerais do projeto.
2. **SUPABASE_KEY**: Você pode usar a **Publishable key** (`sb_publishable_...`) que aparece na tela para as requisições normais do backend. (A Secret Key embaixo é protegida e usada apenas para funções administrativas avançadas que ignoram regras de segurança).

Quer que eu te passe o script SQL completo para você rodar no Supabase e criar todas as tabelas (Igrejas, Acólitos, Missas e Escalas) agora mesmo?

User prompt: essa é a URL?

Response: Quase lá! Essa que você encontrou é a **URL da API REST** (`.../rest/v1/`), que é usada pelo cliente do Supabase para fazer consultas nas tabelas.

Porém, para configurar a conexão base no Python, a biblioteca oficial do Supabase (`supabase-py`) geralmente pede a **Project URL** principal (aquela que aponta para o domínio raiz do projeto, terminando em `.supabase.co` sem o `/rest/v1/`).

Como achar a Project URL correta:

1. No menu lateral esquerdo do Supabase, clique no ícone de engrenagem (**Project Settings**).
2. Vá na aba **API** (ou procure por "Project URL" nas configurações gerais).
3. O formato dela é parecido com isto:

https://rshhejoswuklvgozzgya.supabase.co (tirando o final /rest/v1/).

Se você quiser usar essa URL da imagem no seu .env, você pode usá-la, mas a forma mais padrão para o cliente Python é a URL raiz do projeto.

Quer que eu te mande o script SQL completo para você criar todas as tabelas no Supabase agora que as chaves estão quase prontas?

User prompt: achei. onde exatamente crio o arquivo .env?

Response: O arquivo .env deve ser criado **dentro da pasta backend**, bem ao lado do arquivo main.py que você acabou de testar.

A estrutura das suas pastas no VSCode deve ficar exatamente assim para que o Python consiga ler o arquivo de configuração automaticamente:

```
escala-paroquia/
|
├── backend/
|   ├── venv/
|   ├── .env          <-- CRIE O ARQUIVO AQUI (dentro da pasta backend)
|   └── main.py
└── frontend/
```

Como criar no VSCode:

1. Certifique-se de que a pasta backend está selecionada/aberta.
2. Clique no ícone de **New File** (Novo Arquivo) no topo da barra lateral do VSCode (ou clique com o botão direito em cima da pasta backend e escolha *New File*).
3. Nomeie o arquivo exatamente como **.env** (com o ponto na frente mesmo).
4. Cole suas chaves lá dentro e salve (Ctrl + S).

Depois de criar e salvar o .env, quer que eu te mande o script SQL para você criar as tabelas no Supabase?

User prompt: pronto agora podemos criar o sql

Response: Com o ambiente conectado, vamos criar a estrutura do banco de dados.

O script SQL abaixo foi desenhado exatamente com base na nossa documentação. Ele cria as tabelas na ordem correta (respeitando as chaves estrangeiras), define os tipos de dados e já inclui suportes para os campos de geolocalização e os vínculos de irmãos.

Como rodar no Supabase:

1. Acesse o painel do seu projeto no Supabase.
2. No menu lateral esquerdo, clique no ícone de **SQL Editor** (parece um terminal < >).
3. Clique em **New query**.
4. Copie todo o código SQL abaixo, cole na tela que se abriu e clique no botão verde **Run** (Executar).

```
-- 1. Tabela de Igrejas (Matriz e Capelas)
CREATE TABLE public.igrejas (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  nome_igreja VARCHAR(150) NOT NULL,
  tipo_via VARCHAR(50),
  logradouro VARCHAR(200) NOT NULL,
  numero VARCHAR(20) NOT NULL,
  cep VARCHAR(20) NOT NULL,
  bairro VARCHAR(100),
  lat DECIMAL(10, 8),
  lng DECIMAL(11, 8),
  criado_em TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT
NULL
);

-- 2. Tabela de Acólitos
CREATE TABLE public.acolitos (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  nome_completo VARCHAR(150) NOT NULL,
  classificacao VARCHAR(20) CHECK (classificacao IN ('Responsavel', 'Maior',
'Menor')) NOT NULL,
  grupo_familiar_id INT, -- ID compartilhado para vincular irmãos
  tipo_via VARCHAR(50),
  logradouro VARCHAR(200) NOT NULL,
  numero VARCHAR(20) NOT NULL,
  cep VARCHAR(20) NOT NULL,
  bairro VARCHAR(100),
  lat DECIMAL(10, 8),
  lng DECIMAL(11, 8),
  criado_em TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT
NULL
);

-- 3. Tabela de Missas Padrão (A Rotina Semanal)
CREATE TABLE public.missas_padrao (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  igreja_id UUID REFERENCES public.igrejas(id) ON DELETE CASCADE NOT NULL,
  dia_semana INT CHECK (dia_semana BETWEEN 0 AND 6) NOT NULL, -- 0 = Domingo,
6 = Sábado
  horario TIME NOT NULL,
  vagas_responsavel INT DEFAULT 1 NOT NULL,
  vagas_maior INT DEFAULT 0 NOT NULL,
  vagas_menor INT DEFAULT 0 NOT NULL
);

-- 4. Tabela de Calendário de Exceções (Solenidades e Festas)
```

```

CREATE TABLE public.calendario_excecoes (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    igreja_id UUID REFERENCES public.igrejas(id) ON DELETE CASCADE NOT NULL,
    data_exata DATE NOT NULL,
    horario TIME NOT NULL,
    titulo_evento VARCHAR(150) NOT NULL,
    vagas_responsavel INT DEFAULT 1 NOT NULL,
    vagas_maior INT DEFAULT 0 NOT NULL,
    vagas_menor INT DEFAULT 0 NOT NULL,
    substitui_rotina BOOLEAN DEFAULT TRUE NOT NULL
);

-- 5. Tabela de Escalas Geradas (A Celebração Instanciada)
CREATE TABLE public.escalas_geradas (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    igreja_id UUID REFERENCES public.igrejas(id) ON DELETE CASCADE NOT NULL,
    data_hora TIMESTAMP WITH TIME ZONE NOT NULL,
    tempo_liturgico VARCHAR(100), -- Preenchido automaticamente pelo FastAPI
    mes_ano_referencia VARCHAR(10) NOT NULL, -- Ex: "09-2026" para zerar
    contadores mensais
    status VARCHAR(20) CHECK (status IN ('Rascunho', 'Publicado')) DEFAULT
    'Rascunho' NOT NULL
);

-- 6. Tabela de Relacionamento (Quem serve em qual escala)
CREATE TABLE public.escala_acolitos (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    escala_id UUID REFERENCES public.escalas_geradas(id) ON DELETE CASCADE NOT
    NULL,
    acolito_id UUID REFERENCES public.acolitos(id) ON DELETE CASCADE NOT NULL
);

```

Assim que você rodar esse script no Supabase, todas as tabelas estarão criadas e prontas para receber os dados do nosso backend em FastAPI.

Me avise assim que rodar para darmos o próximo passo!

User prompt: tabelas criadas

Response: Perfeito! Banco de dados estruturado e pronto para uso.

Agora que o esqueleto do banco está criado no Supabase, o próximo passo natural é começarmos a desenvolver os **endpoints (rotas) da nossa API no FastAPI**.

Para que o painel administrativo funcione e o coordenador consiga cadastrar as coisas, precisamos criar as rotas de cadastro (CRUD) para as tabelas principais. Vamos começar pelas **Igrejas** e pelos **Acólitos**.

No seu FastAPI (backend/main.py), vamos estruturar códigos organizados. Para manter o projeto limpo, a

melhor prática é separar a conexão com o Supabase em um arquivo dedicado e depois criar as rotas.

Passo 1: Organizar a Conexão (backend/database.py)

Dentro da pasta backend, crie um arquivo chamado database.py para isolar a inicialização do Supabase:

```
import os
from dotenv import load_dotenv
from supabase import create_client, Client

load_dotenv()

SUPABASE_URL = os.getenv("SUPABASE_URL")
SUPABASE_KEY = os.getenv("SUPABASE_KEY")

if not SUPABASE_URL or not SUPABASE_KEY:
    raise ValueError("As credenciais do Supabase não foram encontradas no
arquivo .env")

supabase: Client = create_client(SUPABASE_URL, SUPABASE_KEY)
```

Passo 2: Criar as Rotas de Acólitos (backend/main.py)

Agora vamos atualizar o seu main.py para usar essa conexão e permitir que a gente cadastre e liste acólitos. Vamos usar modelos do Pydantic (biblioteca padrão do FastAPI) para validar os dados que chegam.

Substitua o código do seu backend/main.py por este:

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Optional
from database import supabase

app = FastAPI(title="API Escala de Acólitos", version="1.0")

# Modelo de dados para validar o cadastro do acólito
class AcolitoCreate(BaseModel):
    nome_completo: str
    classificacao: str # 'Responsavel', 'Maior', 'Menor'
    grupo_familiar_id: Optional[int] = None
    tipo_via: Optional[str] = None
    logradouro: str
    numero: str
    cep: str
    bairro: Optional[str] = None

@app.get("/")
def read_root():
```

```

    return {"status": "API da Paróquia rodando com sucesso!", "banco":
"Conectado"}

# Rota para cadastrar um novo acólito
@app.post("/acolitos/")
def criar_acolito(acolito: AcolitoCreate):
    try:
        # Envia os dados para a tabela 'acolitos' no Supabase
        response = supabase.table("acolitos").insert(acolito.dict()).execute()
        return {"mensagem": "Acólito cadastrado com sucesso!", "dados":
response.data}
    except Exception as e:
        raise HTTPException(status_code=400, data_type=str(e), detail=f"Erro ao
cadastrar: {str(e)}")

# Rota para listar todos os acólitos cadastrados
@app.get("/acolitos/")
def listar_acolitos():
    try:
        response = supabase.table("acolitos").select("*").execute()
        return response.data
    except Exception as e:
        raise HTTPException(status_code=400, detail=f"Erro ao buscar acólitos:
{str(e)}")

```

Como testar:

1. Salve os arquivos. Se o seu terminal com o uvicorn main:app --reload estiver aberto, ele vai recarregar automaticamente.
2. Abra o navegador e acesse: **<http://localhost:8000/docs>**
3. O FastAPI cria **automaticamente** uma documentação interativa fantástica (Swagger UI). Nela, você pode clicar na rota POST /acolitos/, clicar em "Try it out", preencher os dados de teste de um acólito e clicar em "Execute" para ver o dado ser salvo diretamente lá no seu Supabase!

Quer testar essa rota no Swagger e me dizer se conseguiu inserir o primeiro acólito no banco?

User prompt: Me explique melhor como faço os testes

Response: O FastAPI tem uma das melhores ferramentas para testes rápidos já integradas por padrão: a documentação interativa gerada automaticamente (Swagger UI). Ela permite que você teste todas as rotas do seu backend direto pelo navegador, simulando requisições sem precisar escrever nenhum código adicional ou usar programas como o Postman.

Abaixo está o passo a passo para testar as rotas que acabamos de criar.

Step-by-Step: Testando a API no Swagger UI

1. Iniciar o Servidor

Certifique-se de que o servidor do backend está rodando no terminal do VSCode (dentro da pasta backend e com o venv ativo):

```
uvicorn main:app --reload
```

2. Abrir a Documentação Interativa

Abra o seu navegador de preferência e acesse o endereço: <http://localhost:8000/docs>

Você verá uma tela azul/cinza com o título **API Escala de Acólitos** e a lista com as rotas que criamos (GET /, POST /acolitos/, GET /acolitos/).

3. Testar o Cadastro de um Acólito (POST /acolitos/)

1. Clique sobre a barra verde que diz **POST /acolitos/**. A caixa vai se expandir.
2. No canto superior direito da caixa expandida, clique no botão **Try it out** (Testar).
3. Na caixa de texto **Request body**, o FastAPI apresentará um exemplo em formato JSON. Substitua esse texto pelos dados de um acólito real ou de teste:

```
{
  "nome_completo": "Gabriel Silva",
  "classificacao": "Maior",
  "grupo_familiar_id": 1,
  "tipo_via": "Rua",
  "logradouro": "Das Flores",
  "numero": "123",
  "cep": "80000-000",
  "bairro": "Centro"
}
```

4. Clique no botão azul **Execute** logo abaixo.

4. Analisar o Resultado

Role a tela um pouco para baixo até a seção **Responses**:

- **Response body (Sucesso - Código 200/201):** Você verá a mensagem de sucesso e os dados retornados do Supabase, incluindo o id (UUID) criado automaticamente pelo banco.
- **Conferir no Supabase:** Se abrir a aba **Table Editor** no painel do Supabase e clicar na tabela acolitos, a linha referente ao Gabriel Silva estará lá salva.

5. Testar a Busca de Acólitos (GET /acolitos/)

1. Na mesma página do Swagger (<http://localhost:8000/docs>), clique na barra azul **GET /acolitos/**.
2. Clique no botão **Try it out** e em seguida em **Execute**.
3. O sistema fará uma consulta ao Supabase e retornará uma lista contendo todos os acólitos que você já cadastrou até o momento.

Dica de Validação

Se você tentar cadastrar um acólito alterando a "classificacao" para uma palavra não permitida (como "Iniciante"), ou se esquecer de preencher um campo obrigatório como o "logradouro", clique em **Execute** e veja o que acontece. O FastAPI bloqueará o envio e mostrará um erro 422 Unprocessable Entity, explicando exatamente qual campo foi preenchido de forma incorreta.

User prompt: Ele deu um erro ao inciar o servidor

Response: Esse erro acontece porque o terminal fechou ou perdeu a ativação do ambiente virtual (venv) e você também está na pasta raiz do projeto (escala-paroquia), não na pasta do backend. Sem o venv ativo, o Windows não sabe onde encontrar o executável do Uvicorn.

O comando `uvicorn --reload` faz exatamente o mesmo papel de *hot reload* do dotnet watch que apareceu ali na sua tela, reiniciando o servidor a cada alteração no código, mas no ecossistema Python ele exige que o ambiente isolado esteja ligado para reconhecer os pacotes instalados.

Para resolver, rode estes três comandos na sequência no seu terminal:

1. Entre na pasta do backend:
`cd backend`
2. Ative o ambiente virtual (um (venv) verde deve aparecer no início da linha):
`.\venv\Scripts\Activate`
3. Inicie o servidor novamente:
`uvicorn main:app --reload`

Assim que o (venv) aparecer na linha de comando, o terminal voltará a reconhecer o Uvicorn perfeitamente. Conseguiu abrir a tela do Swagger UI para testar o cadastro agora?

User prompt: nao ocorreu isso no meu navegador

Response: A boa notícia é que seu servidor está funcionando perfeitamente e conectado ao banco! O que você está vendo na imagem é a resposta exata da rota raiz (/) que configuramos no código. Isso prova que a API subiu e o Supabase está respondendo.

Para abrir a interface visual de testes (Swagger UI), você só precisa adicionar **/docs** no final do endereço na barra do navegador.

Acesse exatamente este link: <http://127.0.0.1:8000/docs>

Ao carregar essa página, você verá a tela interativa com as opções POST /acolitos/ e GET /acolitos/. Pode

clicar nelas e usar o botão "Try it out" para fazer o seu primeiro cadastro!

User prompt: Deu tudo certo agora. uma pequena duvida como que eu vou colocar para os acólitos que nao tem nenhum vinculo familiar?

Response: Para os acólitos que não têm irmãos servindo na paróquia, você simplesmente deixa o campo vazio. O nosso banco de dados e o modelo do FastAPI já foram configurados para aceitar valores nulos nesse campo de forma nativa.

No momento de testar pelo Swagger UI, você tem duas opções no formato JSON:

- **Apagar a linha:** Simplesmente delete a linha "grupo_familiar_id": 1, do corpo da requisição.
- **Usar null:** Altere o valor numérico para a palavra null (sem aspas), ficando "grupo_familiar_id": null,.

No banco de dados do Supabase, esse registro ficará salvo como NULL. Mais para frente, quando formos programar o motor de sorteio, criaremos uma regra simples: se o campo for nulo, o acólito é escalado individualmente; se possuir um número de identificação, o algoritmo busca os outros acólitos com o mesmo ID para alocá-los na mesma celebração.

Quer cadastrar um acólito sem vínculo para conferir como fica salvo, ou já quer avançar para a criação das rotas de cadastro das Igrejas?
